



Deadlock Verification via Ordering-Constrained Mutex Modeling

Pei Wang^{1,2} , Zhilei Han^{1,2} , Zhihang Sun^{1,2} , and Fei He^{1,2}  

¹ School of Software, BNRist, Tsinghua University,
Beijing 100084, China
{wang-p23,hz121,sunzh20}@mails.tsinghua.edu.cn,
hefei@tsinghua.edu.cn

² Key Laboratory for Information System Security, MoE,
Beijing, China



Abstract. Mutexes are fundamental synchronization primitives in concurrent programming, but their improper use can lead to deadlocks. Conventional assume-based modeling abstracts mutex semantics via assumptions, simplifying safety verification but hindering deadlock verification. Although prior efforts have aimed to address this limitation, we show that state-of-the-art methods remain inaccurate. In this paper, we propose a novel modeling approach that captures mutex semantics using ordering constraints, enabling accurate deadlock verification within partial-order-based concurrent verification frameworks. We formally prove the correctness of our method and implement it in a prototype tool, DEAGLE-DL. We evaluate DEAGLE-DL against a state-of-the-art bounded model checker ESBMC that employs the conventional modeling approach, and a state-of-the-art static analysis tool for deadlock detection. Our experiments show that DEAGLE-DL significantly outperforms both tools in terms of precision, while maintaining substantial efficiency.

1 Introduction

Locks [55, 56, 62] are fundamental synchronization mechanisms in concurrent programs, commonly used to enforce mutual exclusion when accessing critical sections. However, improper use of locks can lead to deadlocks [12, 27, 37–39], a class of concurrency bugs that can severely compromise system reliability. Effective techniques for modeling synchronization primitives and detecting deadlocks are thus essential for building robust concurrent software systems.

This paper focuses on mutexes [8, 25, 53], one of the most widely used synchronization mechanisms in concurrent programming. Verifying mutex operations poses particular challenges due to the waiting behavior associated with lock acquisition. A straightforward solution is to explicitly model the waiting behavior (e.g., using loops), but it significantly increases verification complexity. A widely adopted alternative is assume-based modeling [19, 24, 35, 42, 43, 58], where lock primitives are modeled using `assume` statements. While this method

is effective for safety verification, it is unsound for deadlock verification [43], as it abstracts away the waiting behavior that is essential for identifying deadlocks.

To address this limitation, prior works [13, 43] attempt to augment assume-based modeling with counters that track the number of running and blocked threads. These methods intend to provide a sound framework for deadlock detection. However, they rely heavily on accurate modeling of mutex semantics through updates to counters. Given the intricacies and the volume of subtle cases involved, maintaining precise tracking of blocked threads remains a substantial challenge. It is evidenced by the false positives we first expose in the approaches [13, 43] that utilize these modeling methods, as detailed in Sect. 2.2.

In this paper, we tackle this challenge from a new perspective. The possible behavior of a lock operation is either *immediate acquisition*, *eventual acquisition*, or *indefinite waiting* (see Sect. 3.1 for details). Our key insight is that both eventual acquisition and indefinite waiting violates the **assume** statements used in assume-based modeling, yet they are semantically distinct, because only indefinite waiting constitutes a true deadlock. It reveals a core limitation of assume-based modeling, in that it is unable to differentiate between eventual acquisition and indefinite waiting, which is essential for deadlock verification.

The basic idea of our approach is that, if executions with eventual acquisitions could be excluded, a lock request in the remaining executions then results in either immediate acquisition or indefinite waiting. Therefore, by checking whether any **assume** statement in the assume-based modeling of lock operations is violated, we can precisely identify indefinitely waiting lock requests and thus recognize true deadlocks. To efficiently identify and exclude such executions, we introduce *eventual acquisition constraint* (see Sect. 4.3), an ordering constraint that precisely encodes eventual acquisition. We then incorporate this constraint into our partial-order-based concurrent program verification framework to accurately eliminate such executions. The result is an accurate model of lock primitives without explicitly modeling their waiting behaviors.

Based upon our modeling, we present a deadlock verification method (see Sect. 4.4) that reduces the deadlock verification problem to checking satisfiability of an SMT formula. We formally prove that the formula is satisfiable if and only if the program contains a deadlock.

We implemented our approach in a prototype tool, DEAGLE-DL, built atop DEAGLE [23, 24]. We evaluate DEAGLE-DL against two state-of-the-art tools: a bounded model checker called ESBMC [19] that employs the conventional assume-based modeling, and a state-of-the-art static analysis tool [34] for deadlock detection. We evaluate our approach using two benchmark suites: classical benchmarks and SV-Benchmarks. DEAGLE-DL demonstrates superior performance over the baselines across both benchmark suites. In particular, one previously *unknown* bug in a SV-Benchmarks has been detected by our approach (see Sect. 5.3), which was missed by the baselines.

The primary contributions of this paper are summarized as follows:

1. We reveal that the state-of-the-art methods that utilize assume-based modeling for deadlock detection produce false positives.

2. We devise a novel modeling method for mutex that is based on eventual acquisition constraint. It precisely captures mutex semantics without explicitly modeling its waiting behavior.
3. We propose an efficient deadlock verification approach that integrates our modeling into a partial-order-based concurrent verification framework.
4. We implement a prototype tool, and experimental results show competitive performance of our tool compared with state-of-the-art tools.

2 Existing Approaches for Modeling Mutexes

This section reviews existing modelings of mutexes and highlights their limitations. In particular, we expose a critical flaw of the prevailing assume-based modeling with counters: it can introduce false positives in deadlock detection.

We begin with an informal introduction to mutexes and deadlocks, deferring formal definitions to Sect. 4.1. A mutex m is in either a *locked* ($m = 1$) or *unlocked* ($m = 0$) state, and can only be modified by the **lock** and **unlock** operations. A **lock** operation acquires the mutex if it is unlocked; otherwise, the thread blocks and waits. A **unlock** operation unblocks one waiting thread if any exist; otherwise, it sets the mutex to unlocked. A *deadlock* is a situation in which at least one thread is blocked by a lock operation for a mutex that is never subsequently unlocked by the thread currently holding it [13, 37, 43].

2.1 Assume-Based Modeling

Assume-based modeling, illustrated below, is adopted in many verification techniques [19, 24, 35, 42, 43, 58]. Its core idea is to use an **assume** statement to model **lock**. It avoids the main complexity of modeling the waiting behavior, i.e., when a mutex is already locked, the thread attempting to acquire it must *wait* until the mutex is released. Instead, it only admits executions where the lock is acquired immediately and discards all others, effectively ignoring waiting behavior.

$$\begin{aligned}\text{lock}(\&m) &\stackrel{\text{def}}{=} \text{atomic}\{\text{assume}(m == 0); m = 1; \} \\ \text{unlock}(\&m) &\stackrel{\text{def}}{=} \text{atomic}\{\text{assert}(m == 1); m = 0; \}\end{aligned}$$

An execution is said to be *wait-free* if it contains no attempts to lock a mutex that is already in the locked state. The soundness of the modeling relies on the assumption that any errors appearing in a non-wait-free execution appear also in an equivalent¹ wait-free execution [43]. While this assumption holds for many safety properties, it does not extend to properties like deadlock freedom, which fundamentally depend on the interleaving and waiting semantics of mutexes. As a result, assume-based modeling is inherently *unsound* for deadlock verification.

¹ More precisely, *mutex-free-equivalent*, meaning the execution reaches the same states when the internal behavior of mutexes is disregarded.

2.2 Assume-Based Modeling with Counters

To address the limitations of assume-based modeling, an enhanced approach incorporating counters has been proposed [13, 43]. The core idea is to use two global counters to track the number of running and blocked threads, respectively. By comparing counts, a deadlock is reported if the number of blocked threads equals the number of running threads. However, this approach requires precise tracking of the number of running and blocked threads throughout the program execution, which depends heavily on the scheduling behavior of the underlying operating system. Accurately modeling such behavior is highly challenging.

The modeling of this approach [13, 43] is shown below, where *bcnt* and *rcnt* are two global counters that track the numbers of blocked and running threads, respectively. While the modeling of `unlock` remains unchanged, the modeling of `lock` is split into two atomic blocks. The first atomic block checks whether the target mutex is already locked. If not, it locks the mutex; otherwise, it considers the thread as blocked and increments *bcnt* by 1. The second atomic block checks whether the mutex has since been unlocked. If so, the thread is considered to have recovered from the blocked state, and *bcnt* is decremented by 1. This block also asserts that *bcnt* < *rcnt*. If the assertion fails, a deadlock is reported.

$$\begin{aligned} \text{lock}(\&m) &\stackrel{\text{def}}{=} \left\{ \begin{array}{l} \text{atomic} \left\{ \begin{array}{l} ul = (m == 0); \\ \text{if}(ul) \ m = 1; \text{else } bcnt++; \end{array} \right\} \\ \text{atomic} \left\{ \begin{array}{l} \text{if}(m == 0) \ bcnt--; \\ \text{if}(!ul) \ \left\{ \begin{array}{l} dlf = bcnt < rcnt; \\ \text{assert}(dlf); \\ \text{assume}(!dlf) \end{array} \right\} \end{array} \right\} \end{array} \right\} \\ \text{unlock}(\&m) &\stackrel{\text{def}}{=} \text{atomic}\{\text{assert}(m == 1); m = 0;\} \end{aligned}$$

Importantly, other threads can be scheduled before the second atomic block is reached, therefore an arbitrary number of instructions may occur between the two atomic blocks. We observe that this can lead to an overestimation of the number of blocked threads. For example, if the mutex has already been released, but the blocked thread has not yet reached the second block. Such overestimation of *bcnt* can lead to *false positives* in deadlock detection.

Consider the deadlock-free example and an illustration of its false positive execution² under this modeling in Fig. 1. To make it easier to understand, we use `lock().fir` and `lock().sec` to represent the two atomic blocks involved in a single lock operation. Initially, *bcnt* and *rcnt* are both 0. First, thread t_1 holds mutex m (line 1 – 2), and thread t_2 holds mutex n (line 3 – 4). The counter *rcnt* is incremented to 2. Next, thread t_2 executes the first atomic block within `lock(&m)` (line 5). It attempts to lock mutex m but fails, incrementing *bcnt* to

² We confirm that the current implementation (v8.2) of ESBMC exhibits this false positive. Though it has evolved over the years, its underlying principles are unchanged.

1. Then, thread t_1 unlocks mutex m (line 6). Subsequently, thread t_1 executes the first atomic block within `lock(&n)` (line 7). It attempts to lock mutex n but fails, incrementing $bcnt$ to 2. Finally, thread t_1 executes the second atomic block within `lock(&n)` (line 8), and finds that the assertion $bcnt < rcnt$ is violated because both $bcnt$ and $rcnt$ are equal to 2. As a result, the program terminates and incorrectly concludes that a deadlock has occurred. However, it is clearly a false positive, as t_2 should no longer be blocked once t_1 unlocks m .

t_1	t_2		t_1	t_2
<code>lock(&m);</code>	<code>lock(&n);</code>	$\Rightarrow$	1	<code>lock(&m).fir</code>
<code>unlock(&m);</code>	<code>lock(&m);</code>		2	<code>lock(&m).sec</code>
<code>lock(&n);</code>	<code>unlock(&m);</code>		3	
<code>unlock(&n);</code>	<code>unlock(&n);</code>		4	<code>lock(&n).fir</code>
			5	<code>lock(&n).sec</code>
			6	<code>lock(&m).fir</code>
			7	<code>unlock(&m)</code>
			8	<code>lock(&n).fir</code>
				<code>lock(&n).sec</code>

Fig. 1. A false positive execution (right) of a deadlock-free program (left) under assume-based modeling with counters.

3 The Challenge of Distinguishing Eventual Acquisitions

In this section, we examine the behavior of mutexes from a semantic perspective and show that the central challenge in deadlock verification is to efficiently identify lock operations whose acquisition is eventual.

3.1 Three Possible Outcomes of a Lock Operation

Let m be a mutex variable, and assume executions are fair. A lock operation `lock(&m)` performed by a thread t can behave in one of following ways:

- *Immediate Acquisition*: If m is currently in the unlocked state (i.e., $m = 0$), thread t immediately acquires the mutex and sets m to the locked state.
- *Eventual Acquisition*: If m is currently locked (i.e., $m = 1$) but will eventually be released, thread t waits for a while and then successfully acquires m .
- *Indefinite Waiting*: If m is locked and never released, thread t waits indefinitely, resulting in a *deadlock*.

The two threads in Fig. 2a illustrate an eventual acquisition scenario. The right thread, denoted t_2 , attempts to acquire m , which is currently held by the left thread, denoted t_1 . Here, st is used to denote an arbitrary statement. Regardless of whether t_2 's `lock` occurs before, after or in between the execution of t_1 's st , thread t_2 must wait until t_1 's `unlock` completes. From the perspective of observable effects, these cases are equivalent to the immediate acquisition case shown in Fig. 2b, where t_2 performs its `lock` only after t_1 performs its `unlock`.

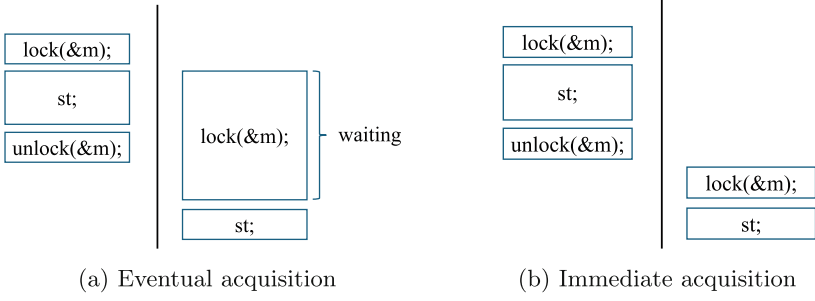


Fig. 2. Eventual acquisition and the corresponding immediate acquisition.

3.2 The Central Challenge

In assume-based modeling (Sect. 2.1), the **assume** statement captures the expectation that the mutex is available at the time of acquisition. Among the three outcomes of a lock operation, only the first (immediate acquisition) satisfies the **assume** condition. The other two (eventual acquisition and indefinite waiting) both violate the assumption, but with different implications. Specifically, only the third case corresponds to a *true* deadlock, while the second involves merely a temporary waiting. Therefore, detecting a violation of the **assume** statement is insufficient for determining whether a deadlock has occurred.

We now consider the program in Fig. 2 under assume-based modeling. For clarity, we use ①, ②, and ③ to denote, respectively, the modeling of **lock**, the statement *st*, and the modeling of **unlock** in thread t_1 . Similarly, ④ and ⑤ denote the modeling of **lock** and the statement *st* in thread t_2 . Without loss of generality, assume that t_1 's **lock** occurs before t_2 's **lock**. A key advantage of assume-based modeling is that mutex operations are represented as atomic operations. This simplifies reasoning about executions: an execution becomes merely an interleaving of these atomic statements. Assuming that *st* is also atomic, all three following interleavings are valid under assume-based modeling (noting that ⑤ becomes unexecutable if the **assume** statement in ④ is violated): ①④②③, ①②④③, and ①②③④⑤. The first two correspond to the eventual acquisition scenario in Fig. 2a, while the last corresponds to the immediate acquisition case depicted in Fig. 2b. If we were to detect deadlocks with violations of the **assume** statement, the first two interleavings would be classified as deadlocks.

However, as previously discussed, an eventual acquisition does not constitute a deadlock, and therefore the deadlock is spurious. Consequently, executions with eventual acquisitions should be excluded from deadlock analysis. It is safe because an eventual acquisition is always equivalent in effect to an immediate acquisition, as shown in Fig. 2. If such executions can be effectively identified and filtered out, such as the first two interleavings discussed above, then a violation of the **assume** statement becomes a sufficient condition for deadlock detection.

The central challenge, therefore, is how to efficiently identify and exclude executions involving eventual acquisitions. This problem is highly non-trivial

Basic domains	Expressions and Commands
$a \in \mathbf{a}$ Local Variables	$\text{EXPR} \ni e ::= v \mid a \mid e \text{ op } e$
$x \in \mathbf{x}$ Shared Variables	$\text{COM} \ni c ::= \text{skip} \mid c \mid x := a \mid a := e \mid a := x$
$m \in \mathbf{m}$ Mutexes	$\mid \text{if } (e) \text{ then } c_1 \text{ else } c_2 \mid c_1; c_2$
$v \in \mathbf{v}$ Values	$\mid \text{assume}(e) \mid \text{assert}(e) \mid \text{atomic}\{c\}$
$t \in \text{TID}$ Thread IDs	$\mid \text{lock}(\&m) \mid \text{unlock}(\&m)$
Programs	
$P \in \text{PROG} \triangleq \text{TID} \rightarrow \text{COM}$	

Fig. 3. Syntax of the Language.

due to the infinite, or at least prohibitively large, execution space. Our solution is to formally encode the eventual acquisition constraint and incorporate it into the analysis, thereby systematically filtering out such executions.

4 Ordering-Constrained Mutex Modeling

In this section, we propose a novel mutex modeling approach based on ordering constraints. We then develop a deadlock verification method that integrates this modeling into a partial-order-based concurrent verification framework. Finally, we formally establish the correctness of the proposed approach.

4.1 Formal Definitions

We use the imperative language shown in Fig. 3 to illustrate our approach, where a concurrent program P is a mapping from each thread to its command. The syntax does not include loop statements, since we assume that loops in the input program have already been handled by other means, such as loop unrolling. For convenience, we assume that the local variables of all threads are disjoint.

A *data state* $s : \mathbf{m} \cup \mathbf{x} \cup \mathbf{a} \rightarrow \mathbf{v}$ is a valuation over $\mathbf{m} \cup \mathbf{x} \cup \mathbf{a}$. A thread $t \in \text{TID}$ is said to be *blocked* if it attempts to acquire a mutex that is currently held by some thread. Note that a single locked mutex may block multiple threads simultaneously. To record this information, we introduce the notion of a *blocking state*, which captures, for each mutex, the sequence of threads currently blocked by it. Formally, a blocking state is a mapping $b : \mathbf{m} \rightarrow \text{TID}^*$, where $b(m)$ is the sequence of threads blocked on mutex m . We use ϵ to denote the empty sequence and $\cdot$ denotes sequence concatenation.

An *environment* is a pair (s, b) of a data state s and a blocking state b . A thread t is *blocked* under an environment (s, b) if there exists a mutex $m \in \mathbf{m}$ and an index i such that the i -th element of the sequence $b(m)$ is t , i.e., $t = b(m)[i]$. We introduce the predicate $B(t)$ to indicate whether thread t is *blocked*.

A *thread-local configuration* is a pair $\langle c, (s, b) \rangle$, where $c \in \text{COM}$ is a command and (s, b) is an environment. A thread-local small-step transition $\langle c, (s, b) \rangle \xrightarrow{e} \langle c', (s', b') \rangle$ is labeled with an event e . An event label may be one of the following: $R(t, x, v)$, denoting that thread t reads value v from shared variable x ; $W(t, x, v)$,

denoting that thread t writes value v to shared variable x ; $L(t, m)$, denoting that thread t attempts to acquire mutex m ; $U(t, m)$, denoting that thread t releases mutex m ; or $\epsilon(t)$, denoting that thread t performs a silent transition that does not access shared variables or mutexes. Let $tid(e)$ denote the thread from which e originates. The small-step semantics of **lock** and **unlock** is shown in Fig. 4. Other commands follow the standard operational semantics, with the sole exception that any transition in thread t requires $\neg B(t)$.

$$\begin{array}{c}
\text{LOCK-ACQUIRE} \frac{\neg B(t) \quad s(m) = 0}{\langle \text{lock}(\&m), (s, b) \rangle \xrightarrow{L(t, m)} \langle \text{skip}, (s[m \mapsto 1], b) \rangle} \\
\text{LOCK-WAIT} \frac{\neg B(t) \quad s(m) = 1}{\langle \text{lock}(\&m), (s, b) \rangle \xrightarrow{L(t, m)} \langle \text{skip}, (s, b[m \mapsto b(m) \cdot t]) \rangle} \\
\text{UNLOCK-AWAIT} \frac{\neg B(t) \quad s(m) = 1 \quad b(m) = t' \cdot \mathbb{L}}{\langle \text{unlock}(\&m), (s, b) \rangle \xrightarrow{U(t, m)} \langle \text{skip}, (s, b[m \mapsto \mathbb{L}]) \rangle} \\
\text{UNLOCK-NOAWAIT} \frac{\neg B(t) \quad s(m) = 1 \quad b(m) = \epsilon}{\langle \text{unlock}(\&m), (s, b) \rangle \xrightarrow{U(t, m)} \langle \text{skip}, (s[m \mapsto 0], b) \rangle} \\
\text{INTERLEAVING} \frac{P(t) = c \quad tid(e) = t \quad \langle c, (s, b) \rangle \xrightarrow{\epsilon} \langle c', (s', b') \rangle}{\langle P, (s, b) \rangle \xrightarrow{\epsilon} \langle P[t \mapsto c'], (s', b') \rangle}
\end{array}$$

Fig. 4. Semantics of lock, unlock and interleaving.

A *global configuration* is a pair $\langle P, (s, b) \rangle$, such that for each thread $t \in \text{TID}$, $\langle P(t), (s, b) \rangle$ is the thread-local configuration of t . The interleaving semantics, shown in Fig. 4, defines a transition relation over global configurations, where each step selects one thread and advances its command while updating the shared environment accordingly.

A thread-local configuration $\langle c, (s, b) \rangle$ is *stuck* if there exists no configuration $\langle c', (s', b') \rangle$ such that $\langle c, (s, b) \rangle \xrightarrow{\epsilon} \langle c', (s', b') \rangle$ is derivable by some transition rule. In other words, a stuck local configuration cannot make any progress. A global configuration is *stuck* if all of its thread-local configurations are stuck. A global configuration is *deadlocked* if it is stuck and at least one thread is blocked in that configuration.

An *execution* is a sequence of transitions over global configurations, denoted by $\langle p_0, (s_0, b_0) \rangle \xrightarrow{e_1} \langle p_1, (s_1, b_1) \rangle \xrightarrow{e_2} \dots \xrightarrow{e_n} \langle p_n, (s_n, b_n) \rangle$, that satisfies the following conditions: (1) for all natural number $i < n$, $\langle p_i, (s_i, b_i) \rangle \xrightarrow{e_{i+1}} \langle p_{i+1}, (s_{i+1}, b_{i+1}) \rangle$ is derivable by some transition rule; (2) the final configuration $\langle p_n, (s_n, b_n) \rangle$ is stuck; and (3) the transitions corresponding to commands within an atomic block occur consecutively. An execution witnesses a *deadlock* if its final configuration is deadlocked.

t_1		t_2	
①: ato	$\left\{ \begin{array}{l} \text{assume}(m_1 == 0); \\ m_2 = 1; \end{array} \right\}$	③: ato	$\left\{ \begin{array}{l} \text{assume}(m_5 == 0); \\ m_6 = 1; \end{array} \right\}$
②: mic	$\left\{ \begin{array}{l} \text{assert}(m_3 == 1); \\ m_4 = 0; \end{array} \right\}$	④: mic	$\left\{ \begin{array}{l} \text{assert}(m_7 == 1); \\ m_8 = 0; \end{array} \right\}$

Fig. 5. SSA form of the two-threaded example.

4.2 Happens-Before Orders

For a program P , given an execution $\langle p_0, (s_0, b_0) \rangle \xrightarrow{e_1} \langle p_1, (s_1, b_1) \rangle \xrightarrow{e_2} \dots$, we can induce some order relations over its events $e_1, e_2, \dots$, which are used in ordering constraints that characterize a certain subset of executions:

- The *program order* relation $\prec_{po}$ is a total order of events from the same thread. Intuitively, $e_1 \prec_{po} e_2$ means that e_1 occurs earlier than e_2 and they are issued by the same thread.
- The *atomicity* relation $\approx$ is an equivalence relation. Intuitively, $e_1 \approx e_2$ means that e_1 and e_2 are contained in the same atomic block.
- The *write serialization* relation $\prec_{ws}$ is a total order among write events that access the same address. Intuitively, $e_1 \prec_{ws} e_2$ iff a write event e_1 occurs earlier than a write event e_2 and they access the same address.
- The *read-from* relation $\prec_{rf}$ links a write event e_1 to a read event e_2 . Intuitively, $e_1 \prec_{rf} e_2$ iff e_2 reads the value written by e_1 .
- The *from-read* relation $\prec_{fr}$ is derived from the $\prec_{ws}$ and $\prec_{rf}$ relations. Specifically, given a pair of write events e_1 and e_2 , and a read event e_3 . If $e_1 \prec_{ws} e_2$ and $e_1 \prec_{rf} e_3$, then the $\prec_{fr}$ relation ensures that e_3 happens before e_2 , i.e., $e_3 \prec_{fr} e_2$, which prevents e_3 from reading from e_2 .

4.3 Eventual Acquisition Constraints

Consider an example consisting of two threads, each containing a lock operation followed by an unlock operation. For clarity, we omit all non-synchronization statements and focus solely on the synchronization behavior. The static single assignment (SSA) form of this example under assume-based modeling is shown in Fig. 5, where each occurrence of a shared variable or mutex – regardless of whether it is read or written – is replaced with a fresh SSA variable.

Given an SSA variable x_i , we write $\langle x_i \rangle$ for its corresponding access event [23]. Let $\mathbb{E}$ represent the set of all events issued by the program. If the type of event is known, we write $\langle x_i \rangle^w$ for a write access and $\langle x_i \rangle^r$ for a read access. Consider the example program in Fig. 5, $\langle m_1 \rangle^r, \langle m_3 \rangle^r, \langle m_5 \rangle^r$ and $\langle m_7 \rangle^r$ represent read accesses, while $\langle m_2 \rangle^w, \langle m_4 \rangle^w, \langle m_6 \rangle^w$ and $\langle m_8 \rangle^w$ represent write accesses.

Without loss of generality, assume that t_1 's lock operation precedes that of t_2 . Figure 6 illustrates the ordering of access events involved in the synchronization

operations ①, ② and ③. Since t_1 acquires the mutex, it retains ownership until it explicitly releases it at ②. As a result, the read access m_3 must read the value written by m_2 . We denote this read-from relation between m_2 and m_3 in Fig. 6 using a solid edge. The read access m_5 , which appears in t_2 's lock operation, may obtain its value from either m_2 or m_4 . If m_5 reads from m_2 , the **assume** statement in ③ fails; however, since t_1 will eventually release the mutex, this scenario corresponds to the interleaving ①③②, which is an instance of eventual acquisition (refer to previous discussion in Sect. 3.2) and should be excluded from deadlock verification. In conclusion, if t_1 has acquired the mutex and will release it in the future (i.e., m_3 reads from m_2), then t_2 's lock operation must not read its value from t_1 's lock operation (i.e., m_5 must not read from m_2).

Formally, given a program P , let $\mathbb{E}_L^w$ denote the set of write events on mutexes that are generated by lock operations in the program. For any write event $e \in \mathbb{E}_L^w$, let $\pi_U^r(e)$ be the set of read events that (i) originate from *unlock* operations of the *same* thread as e , and (ii) may read from e . Similarly, let $\pi_L^r(e)$ be the set of read events that (i) originate from *lock* operations of the *other* thread, and (ii) may read from e . Then we define the following constraint:

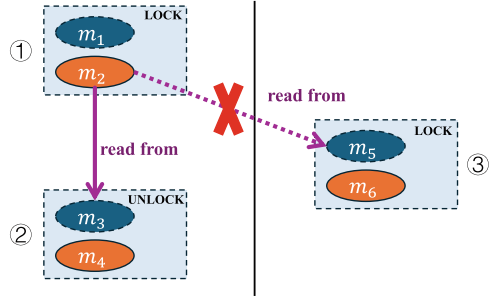


Fig. 6. The orderings of access events: blue represents read, and orange represents write. (Color figure online)

Definition 1 (Eventual Acquisition Constraint). For any write event $e \in \mathbb{E}_L^w$, if there exists a read event $e' \in \pi_U^r(e)$ that reads from e , then for every $e'' \in \pi_L^r(e)$, it must not read from e . Formally,

$$\bigwedge_{e \in \mathbb{E}_L^w} \left(\bigvee_{e' \in \pi_U^r(e)} e \prec_{rf} e' \rightarrow \bigwedge_{e'' \in \pi_L^r(e)} e \not\prec_{rf} e'' \right).$$

Intuitively, if a thread has acquired a mutex (i.e., its lock operation writes to the mutex) and will eventually release it (i.e., a later unlock operation reads from this write), then any lock operation of another thread must not read from that write – such interleavings represent eventual acquisition scenarios, which do not constitute deadlocks and should be excluded from deadlock analysis.

Consider the example, we have $\mathbb{E}_L^w = \{ \langle m_2 \rangle^w, \langle m_6 \rangle^w \}$ and

$$\begin{aligned} \pi_U^r(\langle m_2 \rangle^w) &= \{ \langle m_3 \rangle^r \}, & \pi_L^r(\langle m_2 \rangle^w) &= \{ \langle m_5 \rangle^r \}, \\ \pi_U^r(\langle m_6 \rangle^w) &= \{ \langle m_7 \rangle^r \}, & \pi_L^r(\langle m_6 \rangle^w) &= \{ \langle m_1 \rangle^r \}. \end{aligned}$$

The eventual acquisition constraint is $(\langle m_2 \rangle^w \prec_{rf} \langle m_3 \rangle^r \rightarrow \langle m_2 \rangle^w \not\prec_{rf} \langle m_5 \rangle^r) \wedge (\langle m_6 \rangle^w \prec_{rf} \langle m_7 \rangle^r \rightarrow \langle m_6 \rangle^w \not\prec_{rf} \langle m_1 \rangle^r)$.

4.4 Symbolic Encoding of the Verification Condition

We extend the encoding framework in [1, 18, 23] to support deadlock verification. The framework encodes all possible interleavings of a concurrent program as an SMT formula, thereby reducing the verification task to checking the satisfiability of the resulting formula. Let $guard(e)$ be the guard condition for an event e , and $\langle m_0 \rangle^w$ be the write event corresponding to the initialization of the mutex m .

Value Assignment Encoding. Value assignments of variables in each thread can be encoded by directly interpreting the SSA statements. We use ρ_{va} to represent the encoding. The value assignment encoding of the example is $\rho_{va} := (m_0 = 0) \wedge (m_1 = 0 \rightarrow (m_2 = 1 \wedge m_4 = 0)) \wedge (m_5 = 0 \rightarrow (m_6 = 1 \wedge m_8 = 0))$.

Program Order Constraints. The program order $\prec_{po}$ is a natural order of events from the same thread. We use ρ_{po} to represent the program order constraints. Considering the example, $\rho_{po} := (\langle m_0 \rangle \prec_{po} \langle m_1 \rangle) \wedge (\langle m_0 \rangle \prec_{po} \langle m_5 \rangle) \wedge (\langle m_1 \rangle \prec_{po} \langle m_2 \rangle \prec_{po} \langle m_3 \rangle \prec_{po} \langle m_4 \rangle) \wedge (\langle m_5 \rangle \prec_{po} \langle m_6 \rangle \prec_{po} \langle m_7 \rangle \prec_{po} \langle m_8 \rangle)$.

Atomicity Constraints. Intuitively, $e_1 \approx e_2$ if event e_1 and event e_2 are in the same atomic operation. We use $\rho_{\approx}$ to represent the atomicity constraints. The atomicity constraint of the example is $\rho_{\approx} := (\langle m_1 \rangle \approx \langle m_2 \rangle) \wedge (\langle m_3 \rangle \approx \langle m_4 \rangle) \wedge (\langle m_5 \rangle \approx \langle m_6 \rangle) \wedge (\langle m_7 \rangle \approx \langle m_8 \rangle)$.

Basic Order Constraints. Let $\omega(e)$ be the set of write events that the read event e may read from. For any read $\langle x_i \rangle^r$ and each write $\langle x_j \rangle^w \in \omega(\langle x_i \rangle^r)$ on the same variable $x \in \mathbf{x} \cup \mathbf{m}$, we introduce a Boolean variable $rf_{j,i}^x$, called a *RF variable*, to represent whether event $\langle x_i \rangle^r$ reads from event $\langle x_j \rangle^w$. Similarly, we define a Boolean variable $ws_{i,j}^x$, called a *WS variable*, for each pair of writes $\langle x_i \rangle^w$ and $\langle x_j \rangle^w$ on the same variable $x \in \mathbf{x} \cup \mathbf{m}$, to represent whether $\langle x_i \rangle^w$ occurs earlier than $\langle x_j \rangle^w$. And for each variable $x \in \mathbf{x} \cup \mathbf{m}$, we derive from-read relation from RF and WS variables. We encode these three partial-order relations following the approach of [18, 23]. Let ρ_{rf} the conjunction of all read-from constraints, ρ_{ws} denote the conjunction of all write-serialization constraints, and ρ_{fr} the conjunction of all from-read constraints.

Eventual-Acquisition Constraint. For any write event $\langle m_i \rangle^w \in \mathbb{E}_L^w$, if there is a read $\langle m_j \rangle^r \in \pi_U^r(\langle m_i \rangle^w)$ obtaining its value from $\langle m_i \rangle^w$, then for any read $\langle m_k \rangle^r \in \pi_L^r(\langle m_i \rangle^w)$, it must not read from $\langle m_i \rangle^w$. Since we have already defined the *RF variables* to represent the read-from relations, the eventual-acquisition constraint mentioned in Definition 1 can now be expressed as follows:

$$\rho_{eac} := \bigwedge_{\langle m_i \rangle^w \in \mathbb{E}_L^w} \left(\bigvee_{\langle m_j \rangle^r \in \pi_U^r(\langle m_i \rangle^w)} rf_{i,j}^m \rightarrow \bigwedge_{\langle m_k \rangle^r \in \pi_L^r(\langle m_i \rangle^w)} \neg rf_{i,k}^m \right)$$

Considering the example, the corresponding eventual acquisition constraint can be expressed as: $\rho_{eac} := (rf_{2,3}^m \rightarrow \neg rf_{2,5}^m) \wedge (rf_{6,7}^m \rightarrow \neg rf_{6,1}^m)$.

Potential-Deadlock Constraint. Let $\mathbb{E}_L^r$ be the set of all read events on mutex variables that are generated by lock operations in the program. An execution is flagged as potentially deadlocked if at least one **assume** statement used to model a lock operation is violated, i.e., the following constraint is satisfied:

$$\rho_{dl} := \bigvee_{\langle m_k \rangle_l^r \in \mathbb{E}_L^r} guard(\langle m_k \rangle_l^r) \wedge (m_k = 1)$$

Consider the example program, we have $\mathbb{E}_L^r = \{\langle m_1 \rangle_l^r, \langle m_5 \rangle_l^r\}$. Since there is no control flow statements preceding $\langle m_1 \rangle_l^r$ and $\langle m_5 \rangle_l^r$, their guard conditions are both *true*. So the potential-deadlock constraint ρ_{dl} is:

$$\rho_{dl} := (m_1 = 1) \vee (m_5 = 1)$$

The Whole Encoding Formula. The whole encoding formula for a program is $\Psi := \Phi_{valid} \wedge \rho_{eac} \wedge \rho_{dl}$, where $\Phi_{valid} := \rho_{va} \wedge \rho_{po} \wedge \rho_{\approx} \wedge \rho_{ws} \wedge \rho_{rf} \wedge \rho_{fr}$.

4.5 Correctness

Given a concurrent program P , let P' denote the program obtained by replacing all mutex primitives in P with assume-based modeling. Let $\llbracket P \rrbracket$ be the set of executions of P , and $\llbracket P' \rrbracket$ be the set of executions of P' . We write R_l and R_u to signify specifically the read generated by lock and unlock operations.

Definition 2 (Eventual Acquisition). An execution $\sigma' \in \llbracket P' \rrbracket$ has eventual acquisitions, if $\exists t_1, t_2 \in \text{TID}, t_1 \neq t_2, \exists m \in \mathbf{m}, \sigma'$ has a subsequence of the form:

$$\dots \xrightarrow{W(t_1, m, 1)} \dots \xrightarrow{R_l(t_2, m, 1)} \dots \xrightarrow{W(t_1, m, 0)} \dots$$

and there is no other write to m between $W(t_1, m, 1)$ and $W(t_1, m, 0)$.

Lemma 1. An execution $\sigma' \in \llbracket P' \rrbracket$ violates the eventual acquisition constraint if and only if it has eventual acquisitions.

Proof. From Definition 1, the constraint is violated if and only if σ' has a write $\langle m_i \rangle^w \in \mathbb{E}_L^w$ (i.e., $W(t_1, m, 1)$), satisfying the following conditions:

- $\exists \langle m_j \rangle^r \in \pi_U^r(\langle m_i \rangle^w)$ such that $\langle m_i \rangle^w \prec_{rf} \langle m_j \rangle^r$, i.e., $R_u(t_1, m, 1)$ from an unlock operation reads from $W(t_1, m, 1)$. It means that there is no intervening write to $W(t_1, m, 1)$ and $R_u(t_1, m, 1)$. Moreover, since $R_u(t_1, m, 1)$ is immediately followed by the write $W(t_1, m, 0)$ generated by the same unlock operation, there is no other write to m between $W(t_1, m, 1)$ and $W(t_1, m, 0)$.

- $\exists \langle m_k \rangle^r \in \pi_L^r(\langle m_i \rangle^w)$ such that $\langle m_i \rangle^w \prec_{rf} \langle m_k \rangle^r$, i.e., $R_l(t_2, m, 1)$ from a lock operation ($t_2 \neq t_1$) reads from $W(t_1, m, 1)$. It means that $R_l(t_2, m, 1)$ must occur between $W(t_1, m, 1)$ and $W(t_1, m, 0)$.

So σ' contains the subsequence “ $\dots \xrightarrow{W(t_1, m, 1)} \dots \xrightarrow{R_l(t_2, m, 1)} \dots \xrightarrow{W(t_1, m, 0)}$...” and there is no other write to m between $W(t_1, m, 1)$ and $W(t_1, m, 0)$. Therefore, it has eventual acquisitions. $\square$

By enforcing the eventual acquisition constraint, we systematically eliminate executions involving eventual acquisitions, thereby preserving the intended semantics of lock operations without explicitly modeling waiting behaviors.

Formally, an execution $\sigma' \in \llbracket P' \rrbracket$ is *potentially deadlocked* if $\exists t \in \text{TID}, \exists m \in \mathbf{m}, \sigma'$ has a subsequence of the form “ $\dots \xrightarrow{R_l(t, m, 1)} \dots$ ”. Such an execution lead either to an eventual acquisition or to a deadlock involving indefinite waiting.

We can prove Theorem 1 to ensure the correctness of our modeling.

Theorem 1. *There is an execution $\sigma \in \llbracket P \rrbracket$ that witnesses a deadlock if and only if there is an execution $\sigma' \in \llbracket P' \rrbracket$ such that σ' is flagged as potentially deadlocked and satisfies the eventual acquisition constraint.*

Proof. ($\Leftarrow$): Suppose $\sigma' \in \llbracket P' \rrbracket$ is flagged as potentially deadlocked, i.e., $\exists t_1 \in \text{TID}, \exists m \in \mathbf{m}$ such that σ' has a read $R_l(t_1, m, 1)$; and σ' satisfies the eventual acquisition constraint, i.e., σ' has no eventual acquisitions by Lemma 1. Let the last write to m preceding $R_l(t_1, m, 1)$ be $W(t_2, m, 1)$. We can get an execution $\sigma \in \llbracket P \rrbracket$ by replacing each read and write to a mutex in σ' with the corresponding lock or unlock operation. Let $W(t_2, m, 1)$ in σ' corresponds to $e_j : L(t_2, m)$ in σ , and let $R_l(t_1, m, 1)$ in σ' correspond to $e_k : L(t_1, m)$ in σ , where j and k are the indices in σ . According to the semantics in Fig. 4, after executing $L(t_1, m)$, t_1 is blocked (i.e. $B(t_1)$ holds). If $t_1 = t_2$ (denoted by t), then since the only thread that can unblock t is t itself, $B(t)$ must hold in every environment (s_i, b_i) of σ for all $i > k$. If $t_1 \neq t_2$, no $W(t_2, m, 0)$ occurs after $W(t_2, m, 1)$ in σ' because σ' has no eventual acquisitions. So for all $i > k$, $e_i \neq U(t_2, m)$ in σ , and $B(t_1)$ holds in environment (s_i, b_i) of σ . Therefore, the final configuration (s_n, b_n) of σ is deadlocked, i.e., σ witnesses a deadlock.

($\Rightarrow$): Suppose $\sigma \in \llbracket P \rrbracket$ witnesses a deadlock, i.e., $B(t_1)$ holds in the final configuration (s_n, b_n) of σ . According to the semantics in Fig. 4, the only transition rule that can block a thread is Lock-Wait. Hence, there must exist a lock event $e_k : L(t_1, m)$ in σ that satisfies $s_{k-1}(m) = 1$. As shown in Fig. 2, an eventual acquisition scenario of P can be transformed into the corresponding immediate acquisition scenario by delaying the corresponding lock operation. So we assume without loss of generality that σ contains no eventual acquisitions; formally, no transition in σ is derived using the Unlock-Await rule. By substituting each lock or unlock operation in σ with the corresponding read and write to the mutex, we obtain an execution $\sigma' \in \llbracket P' \rrbracket$. Clearly, σ' contains no eventual acquisitions and thus satisfies the eventual acquisition constraint by Lemma 1. Moreover, the lock event $e_k : L(t_1, m)$ in σ induces a read event $R_l(t_1, m, 1)$ in σ' , which causes σ' to be flagged as potentially deadlocked. $\square$

To establish the correctness of our encoding, we rely on results from prior work [1, 18, 23]: *The formula Φ_{valid} characterizes all valid executions of the program.* Therefore, we derive the following theorem:

Theorem 2. *The formula Ψ is satisfiable if and only if there is an execution $\sigma \in \llbracket P \rrbracket$ that witnesses a deadlock.*

Proof. By prior work [1, 18, 23], $\Psi = \Phi_{valid} \wedge (\rho_{eac} \wedge \rho_{dl})$ is satisfiable iff there is an execution $\sigma' \in \llbracket P' \rrbracket$ such that σ' satisfies the eventual-acquisition constraint ρ_{eac} and the potential-deadlock constraint ρ_{dl} . According to Theorem 1, it means that there is an execution $\sigma \in \llbracket P \rrbracket$ that witnesses a deadlock. $\square$

4.6 Discussion on Extensibility

In principle, our approach can be naturally extended to synchronization primitives that rely on a waiting mechanism (e.g., semaphores, recursive locks, spinlocks, read-write locks, ...). Their behavior could be classified into either immediate success, eventual success, or indefinite waiting. And they could be modeled analogously to eliminate the eventual success case.

We take semaphores as an example to illustrate how our approach can be extended. Let S be a semaphore with N available resources, equipped with an internal counter $S.count$ that records the number of currently available resources. Initially, $S.count = N$. The semaphore provides the following two primitives:

- **sem_wait**(& S): if $S.count = 0$, the calling thread blocks; otherwise, it acquires one unit of the resource and decrements $S.count$;
- **sem_post**(& S): the calling thread releases one unit of the resource, increments $S.count$, and wakes up one blocked thread (if any).

Similar to Sect. 3.1, we can classify a **sem_wait**(& S) operation executed by a thread t into the following three cases:

- *Immediate Success*: If there are currently available resources (i.e., $S.count > 0$), t immediately acquires one unit of the resource and decrements $S.count$;
- *Eventual Success*: If there are currently no available resources (i.e., $S.count = 0$) but a resource will be released in the future, then t waits for a while and then acquires one unit of the resource;
- *Indefinite Waiting*: If there are currently no available resources and no resource will be released, then t waits indefinitely, resulting in a deadlock.

Similar to our discussion in Sect. 3.1, the eventual success does not need to be considered during verification, as it is equivalent to an immediate success. Therefore, it is safe to eliminate the eventual success case during verification. Assume that we model **sem_wait**(& S) as **atomic**{**assume**($S.count > 0$); $S.count - -$; } and **sem_post**(& S) as **atomic**{**assert**($S.count < N$); $S.count + +$; }. Then, we can eliminate all eventual success cases by introducing a constraint similar to the one in Sect. 4.3. Intuitively, it must ensure that for any **sem_wait**(& S) operation that acquires the last available resource (i.e., $S.count = 1$), if it is read by any

`sem_post(&S)` operation from the same thread, then it must not be read by any `sem_wait(&S)` operation from a different thread.

In addition, our core idea (i.e., using ordering constraints to eliminate *eventual acquisition* cases during verification) remains applicable under weak memory models. The main challenge in this extension lies in accurately characterizing valid executions. In fact, the framework on which we build has already extended Φ_{valid} to TSO and PSO weak memory models [18]. Based on these updated Φ_{valid} constraints, our encoding can be transferred to the corresponding weak memory models with only minimal modifications.

5 Implementation and Evaluation

5.1 Implementation, Baseline and Setup

We implemented our approach in a prototype tool, DEAGLE-DL, built on top of DEAGLE [23, 24]. DEAGLE comprises two main components: a frontend and a backend. The frontend, primarily adapted from CBMC [35], models concurrent programs using partial orders, while the backend implements an enhanced SMT solver for verifying concurrent programs. We extend DEAGLE’s frontend by incorporating our proposed modeling approach for mutexes and encoding of potential-deadlock constraints, enabling DEAGLE-DL to conduct automated deadlock verification. DEAGLE-DL handles loops via unrolling. Crucially, it enforces full unrolling for loops inside critical sections. The reason is that a partially unrolled loop within a critical section may omit a subsequent `unlock` operation, causing the mutex to be spuriously left unreleased in the unrolled program, and thereby introducing false deadlocks that do not exist in the original program.

To assess the effectiveness of DEAGLE-DL, we compare it against two representative state-of-the-art tools: a bounded model checker called ESBMC [19] and a static analyzer developed by Daniel Kroening et al. [34].

- ESBMC [19] is a model checker that adopts the assume-based modeling with counters (Sect. 2.2) to support deadlock verification. It represents a state-of-the-art deadlock verifier based on traditional mutex modeling.
- The static analyzer [34] is a state-of-the-art static analysis tool for deadlock detection. It performs context- and thread-sensitive lockset analysis to construct a lock graph, followed by iterative cycle refinement.

All experiments were conducted on a server equipped with two AMD EPYC 7H12 64-core processors, running Ubuntu 20.04.6 LTS. Following the experimental settings of SV-COMP, each verification task was given a time limit of 900s. Both DEAGLE-DL and ESBMC [19] handles loops via unrolling. There are heuristics for selecting loop unrolling depths. For example, users may set the bound based on the maximum number of loop iterations observed in typical executions or test suites. In our evaluations, we adopt a standard strategy that increments the loop unrolling bound following the Fibonacci sequence until either a counterexample is found (reported as *deadlock*) or the unrolling depth is

deemed sufficient (reported as *deadlock-free*). If the time limit is reached before either condition is met, the result is reported as *bounded deadlock-free*. Note that a *bounded deadlock-free* result gives users increased confidence that the program is deadlock-free, but it does not provide a full correctness guarantee. A special case arises when the time limit is reached before the tool completes checking the first unrolling; in this case, the result is reported as *unknown*. For the static analyzer [34], *bounded deadlock-free* results do not apply. If a deadlock is detected, it reports *deadlock*; otherwise, it reports *deadlock-free*. Any unexpected interruption or error is reported as *error*.

Table 1. Results on Classical Benchmarks. The table reports the number of tasks classified as *deadlock* (*dl*), *deadlock-free* (*dlf*), and *bounded deadlock-free* (*bdf*) by each tool, where $\mathbf{x}(\mathbf{y})$ means the tool reported $\mathbf{y}$ results in this category, of which only $\mathbf{x}$ are correct.

Tool	Reported Results			Avg. Costs	
	dl	dlf	bdf	Time (s)	Memory (MB)
DEAGLE-DL	15	15	0	3.7	49
ESBMC	0	0	15(30)	900	27097
STATIC ANALYZER	0(12)	3(18)	0	<0.1	8

5.2 Evaluation on Classical Benchmarks

We conducted experiments on a suite of classical concurrent programming benchmarks [3, 15, 26, 50]. These include the *dining philosophers*, *bank transfer*, *traffic intersection*, *tasks-printers*, and *chefs-appliances* problems. For each benchmark, we considered both deadlocking and deadlock-free variants, resulting in a total of 10 parameterized programs. The former corresponds to the implementation of the original problem, while the latter is obtained by integrating the deadlock-avoidance strategies commonly presented in tutorials into the original versions.

We instantiated these with varying numbers of threads (e.g., 3,5,7), yielding a total of 30 concrete instances. Table 1 presents the results of the tools.

Precision. DEAGLE-DL produces no incorrect results across all instances, outperforming baselines. ESBMC failed to detect any of the 15 deadlocking instances within the allotted time, and also produced only bounded deadlock-free results for the 15 deadlock-free instances. The static analyzer failed to correctly identify any deadlock among the 15 deadlocking instances, which is likely due to imprecision introduced by its overly approximate pointer analysis. For the 15 deadlock-free instances, the static analyzer produced 12 false positives and correctly identified only 3 as deadlock-free. This is largely because the deadlock-avoidance strategies in these classical problems often rely on subtle techniques.

Such non-trivial control flows are difficult to track under the approximations and abstractions adopted by the static analysis tool, often leading to false positives. In contrast, our method comprehensively captures and models these control flows, thereby ensuring accurate reasoning about deadlock.

Efficiency. DEAGLE-DL outperforms ESBMC in both runtime and memory efficiency, but remains less efficient than the static analyzer. ESBMC requires on average nearly 900 s of runtime and 26 GB of memory. In contrast, DEAGLE-DL requires on average only 3.7 s and 49 MB of memory. The advantages of DEAGLE-DL in efficiency can be theoretically explained at the modeling level. As discussed in Sect. 2.2, traditional modeling introduce counters and a large number of auxiliary statements to maintain them for deadlock verification, which leads to significant overhead in both time and space. In contrast, we directly model the effect of waiting by introducing additional eventual acquisition constraints, which allows us to bypass the intricate details involved in modeling the waiting process. Furthermore, since these constraints exclude certain unnecessary executions from the valid execution set, they reduce the total number of executions that need to be explored during verification, which in some cases can further alleviate the time and space burdens.

The static analyzer is highly efficient, requiring no more than 0.1 s and 8 MB across all examples. However, it comes at the cost of extensive approximations and abstractions of the program. As discussed above, such approximations can produce numerous imprecise results and provide comparatively weaker theoretical guarantees. This poses a challenge for complex concurrent programs, where manually inspecting potential deadlocks is far from trivial. In contrast, our method produces no false positives and ensures (bounded) completeness.

5.3 Evaluation on SV-Benchmarks

We conducted our second set of experiments using the *Concurrency* category of the SV-Benchmarks suite [6], specifically the *svcomp25-freeze* version (the frozen release prepared for SV-COMP 2025). Derived from real-world applications and research prototypes, these benchmarks are more complex and representative than classical ones and have been widely adopted in the evaluation of concurrent program verification techniques [1, 6, 18, 23, 45, 54]. We excluded benchmark programs that do not use mutex primitives (e.g., `lock` and `unlock`) or that relied on other synchronization primitives, resulting in a total of 288 benchmark programs. For all experiments, we used the `.i` files provided by SV-Benchmarks, which are the preprocessed versions of the original source files. Since the SV-Benchmarks suite does not include labeled ground truth for deadlock properties, we established the ground truth through cross-referencing and manual inspection.

For each tool evaluated, Table 2 presents the reported results on the SV-Benchmarks, and Table 3 shows the average time and memory usage.

Table 2. Verification results on the SV-Benchmarks. The table lists the number of tasks reported as *deadlock* (*dl*), *deadlock-free* (*dlf*), *bounded deadlock-free* (*bdf*), *unknown* (*unk*), and *error* (*err*) by each tool.

Benchmark Subcategories	#. Tasks	DEAGLE-DL				ESBMC				STATIC ANALYZER		
		dl	dlf	bdf	unk	dl	dlf	bdf	unk	dl	dlf	err
goblint-regression	190	9	110	71	0	6(33)	60	82	15	9(13)	177	0
ldv-races	19	0	17	2	0	0	12	0	7	0	11	8
pthread	19	0	6	13	0	0	0	4	15	0(3)	13	3
pthread-C-DAC	3	0	0	3	0	0	0	1	2	0	3	0
pthread-atomic	4	0	2	2	0	0	0	0	4	0	3	1
pthread-complex	2	0	2	0	0	0	0	2	0	0	2	0
pthread-deagle	2	0	2	0	0	0	0	0	2	0	2	0
pthread-divine	1	0	0	1	0	0	0	0	1	0	1	0
pthread-ext	20	1	0	19	0	0	0	13	7	0	13(14)	6
pthread-lit	2	0	0	2	0	0	0	1	1	0	2	0
pthread-nondet	2	0	0	2	0	0	0	0	2	0	2	0
pthread-race-challenges	24	0	4	20	0	0	0	13	11	0	24	0
Overall	288	10	143	135	0	6(33)	72	116	67	9(16)	253(254)	18

Precision. Table 2 shows that DEAGLE-DL significantly outperforms baselines in terms of the precision of deadlock detection as well as deadlock-free verification. For the 10 benchmark programs that contains deadlocks (9 from the *goblint-regression* and 1 from the *pthread-ext* subcategory), DEAGLE-DL successfully detected all of them without any false positives. For the 278 deadlock-free programs, DEAGLE-DL proved that over half are free of deadlocks, and established bounded deadlock-freedom for the rest. Notably, the deadlock detected by DEAGLE-DL in the *pthread-ext* subcategory revealed a previously *unknown* bug that could potentially influence the final SV-COMP results. This issue is caused by an `unlock` operation that was mistakenly written as a `lock` operation. We discovered this bug through deadlock detection and confirmed via code inspection that it was indeed a coding mistake. We reported this issue to the SV-Benchmarks organizers, and it was subsequently fixed. Interestingly, this bug had gone unnoticed by both ESBMC and the static analyzer.

ESBMC reported 33 deadlocks, of which only 6 were correct. Among the 27 false positives, 4 were caused by the failure to account for the program termination semantics of the return statement in the main function³. The remaining 23 false positives arise from the previously mentioned flaws (Sect. 2.2) of assume-based modeling with counters. In addition, ESBMC misses 4 programs with deadlocks (including the unnoticed deadlock mentioned above) because it

³ When the main function returns, the program terminates immediately, so threads waiting for resources not released by main are not considered a deadlock in practice.

is unable to find corresponding counterexamples within the time limit. At the same time, ESBMC proved fewer deadlock-free (or bounded) results compared with DEAGLE-DL. The static analyzer reported 16 deadlocks, of which only 9 were correct. Notably, among the 254 programs that the static analyzer reported as deadlock-free, only 253 were truly deadlock-free. The missed deadlock corresponds exactly to the unnoticed mistake in SV-Benchmarks.

In addition, we evaluated two dynamic analysis tools on 10 deadlocking examples from the SV-Benchmarks suite: UNDEAD [61] (to our knowledge the most recent open-source dynamic deadlock detector for C programs) and THREAD-SANITIZER [49] (a widely used industrial dynamic analysis tool developed by Google). Since these tools do not recognize some verification-only functions, we manually removed or replaced them (e.g., replacing `__VERIFIER_nondet_int()` with `random()`). Within the given time limit, these tools continuously sample execution traces for analysis until a deadlock is detected. They were both able to detect deadlocks in only 3 out of the 10 programs. Notably, neither tool detected the previously unknown bug discovered by our tool.

Efficiency. Table 3 shows that DEAGLE-DL demonstrates substantial improvements in efficiency within bounded model checking, though the static analysis method remains the most efficient overall. Compared with ESBMC, DEAGLE-DL achieves on average a $1.5\times$ improvement in runtime and reduces memory consumption significantly. In contrast, the static analysis tool [34] still outperforms both bounded model checking tools, DEAGLE-DL and ESBMC, in terms of efficiency. However, as discussed earlier, this efficiency of the static analysis approach is achieved at the cost of imprecise results and weaker theoretical guarantees, stemming from the extensive approximations and abstractions it relies on.

Table 3. Average Costs on SV-Benchmarks.

Tool	Time (s)	Mem (GB)
DEAGLE-DL	384	2.26
ESBMC	593	25.26
STATIC ANALYZER	<1	<1

6 Related Work

Static deadlock detection analyzes the source code of the program itself rather than specific executions. The key benefit lies in its potential to exhaustively examine all executions to identify potential deadlocks. There are plenty of researches focusing on static deadlock detection:

Dataflow-analyses-based techniques track relations between threads and usage of resources, building an approximation of a dependency graph. Representative tools include LOCKLINT [14], which requires user annotations; RACERX [16], which applies lockset analysis without pointer analysis; INFER [21], a compositional but context-insensitive analyzer; and PEAHEN [11], which constructs context-insensitive lock graphs and incrementally refines the deadlock

cycles. Brotherston et al. [7] propose a compositional static deadlock detector with abstract interpretation for Android Java, which is designed for code-review-time and incremental analysis over large industrial codebases, prioritizing scalability. Kroening et al. [34] proposes a context- and thread-sensitive static analyzer, which speeds up pointer analysis with dependency analysis, and reduces false positives with non-concurrency graph analysis. While these approaches gain efficiency from approximation and abstraction, they tend to generate some false positives. On the other hand, our approach offers stronger theoretical guarantees.

Bounded-model-checking-based techniques have recently shown strong potential for verifying concurrent programs. Alglave et al. [1, 2] propose a partial-order-based bounded verification framework. He et al. [18, 23, 54] have enhanced this framework by deeming some relations unnecessary and eliminating them. Other researchers have further developed and applied it, as seen in projects like YOGAR-CBMC [58–60], LAZY-CSEQ [30], DARTAGNAN [20, 36], and more. TCBMC [43] first introduced the assume-based modeling of mutexes. It further proposed the incorporation of counters within assume-based modeling to determine whether deadlock occurs. ESBMC [13] refines it, considering additional scenarios and details to enhance the accuracy of counters. Our approach requires no consideration of the intricate details as in the aforementioned modeling, and achieves deadlock verification in a natural and straightforward manner, yielding not only more precise results but also significantly improved efficiency.

Dynamic deadlock detection focuses on analyzing concrete executions of a program rather than its source code. These techniques have sought to combine testing with formal reasoning to address the inherent limitations of purely testing-based approaches. For instances, the classic GoodLock algorithm [22] has inspired a series of improvements [5, 9, 49, 61], some of which employ re-execution strategies to confirm potential deadlocks [4, 31, 46, 47, 52]. Predictive analyses infer bugs in alternate executions [28, 29, 33, 40, 41, 44, 48, 51] and avoid false positives, which has effectively detected deadlocks [10, 17, 32].

7 Conclusion

In this paper, we propose a novel approach to model mutexes, which captures lock semantics while avoiding the need to explicitly model waiting behavior. Our modeling addresses issues that many complex patches in previous work could not fully resolve, by introducing a simple constraint, termed the eventual acquisition constraint. We further propose a deadlock verification method and prove its theoretical correctness. We have implemented our method in a prototype tool. It demonstrates the competitive performance compared to state-of-the-art tools.

Acknowledgments. We thank the anonymous reviewers for their insightful comments, which improved the presentation of this paper. This work was supported by the National Natural Science Foundation of China (No. 62072267 and No. 62021002).

Data Availability Statement. The artifact contains the source code, executable binaries, scripts, benchmark data, and result-processing scripts required to reproduce the experimental results presented in our paper. It is available on Zenodo [57].

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alglave, J., Kroening, D., Tautschnig, M.: Partial Orders for Efficient Bounded Model Checking of Concurrent Software. In: Sharygina, N., Veith, H. (eds.) CAV 2013. LNCS, vol. 8044, pp. 141–157. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39799-8_9
2. Alglave, J., Maranget, L., Sarkar, S., Sewell, P.: Fences in Weak Memory Models. In: Touili, T., Cook, B., Jackson, P. (eds.) CAV 2010. LNCS, vol. 6174, pp. 258–272. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-14295-6_25
3. Andrews, G.R.: Concurrent programming: principles and practice. Benjamin-Cummings Publishing Co., Inc (1991)
4. Bensalem, S., Fernandez, J.C., Havelund, K., Mounier, L.: Confirmation of deadlock potentials detected by runtime analysis. In: Proceedings of the 2006 Workshop on Parallel and Distributed S: Testing and Debugging, pp. 41–50 (2006). <https://doi.org/10.1145/1147403.1147412>
5. Bensalem, S., Havelund, K.: Dynamic Deadlock Analysis of Multi-threaded Programs. In: Ur, S., Bin, E., Wolfsthal, Y. (eds.) HVC 2005. LNCS, vol. 3875, pp. 208–223. Springer, Heidelberg (2006). https://doi.org/10.1007/11678779_15
6. Beyer, D., Strejček, J.: Improvements in software verification and witness validation: SV-COMP 2025. In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, pp. 151–186. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-90660-2_9
7. Brotherston, J., Brunet, P., Gorogiannis, N., Kanovich, M.: A compositional deadlock detector for android java. In: 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 955–966. IEEE (2021). <https://doi.org/10.1109/ASE51524.2021.9678572>
8. Butenhof, D.R.: Programming with POSIX threads, Addison-Wesley Professional (1993)
9. Cai, Y., Meng, R., Palsberg, J.: Low-overhead deadlock prediction. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 1298–1309 (2020). <https://doi.org/10.1145/3377811.3380367>
10. Cai, Y., Yun, H., Wang, J., Qiao, L., Palsberg, J.: Sound and efficient concurrency bug prediction. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 255–267 (2021). <https://doi.org/10.1145/3468264.3468549>
11. Cai, Y., Ye, C., Shi, Q., Zhang, C.: Peahen: Fast and precise static deadlock detection via context reduction. In: Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 784–796 (2022). <https://doi.org/10.1145/3540250.3549110>
12. Coffman, E.G., Elphick, M., Shoshani, A.: System deadlocks. ACM Comput. Surv. (CSUR) **3**(2), 67–78 (1971). <https://doi.org/10.1145/356586.356588>

13. Cordeiro, L., Fischer, B.: Verifying multi-threaded software using SMT-based context-bounded model checking. In: Proceedings of the 33rd International Conference on Software Engineering, pp. 331–340 (2011). <https://doi.org/10.1145/1985793.1985839>
14. Corporation, O.: Locklint overview. <https://docs.oracle.com/cd/E19059-01/wrkshp50/805-4947/6j4m8jrng/index.html>
15. Dijkstra, E.W.: Cooperating sequential processes. In: The origin of concurrent programming: from semaphores to remote procedure calls, pp. 65–138. Springer, Cham (2002). https://doi.org/10.1007/978-1-4757-3472-0_2
16. Engler, D., Ashcraft, K.: RacerX: Effective, static detection of race conditions and deadlocks. *ACM SIGOPS Oper. Syst. Rev.* **37**(5), 237–252 (2003). <https://doi.org/10.1145/945445.945468>
17. Eslamimehr, M., Palsberg, J.: Sherlock: scalable deadlock detection for concurrent programs. In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, pp. 353–365 (2014). <https://doi.org/10.1145/2635868.2635918>
18. Fan, H., Sun, Z., He, F.: Satisfiability modulo ordering consistency theory for SC, TSO, and PSO memory models. *ACM Trans. Program. Lang. Syst.* **45**(1), 1–37 (2023). <https://doi.org/10.1145/3579835>
19. Gadelha, M.R., Monteiro, F.R., Morse, J., Cordeiro, L.C., Fischer, B., Nicole, D.A.: ESBMC 5.0: an industrial-strength C model checker. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, pp. 888–891 (2018). <https://doi.org/10.1145/3238147.3240481>
20. Gavrilenko, N., Ponce-de-León, H., Furbach, F., Heljanko, K., Meyer, R.: BMC for Weak Memory Models: Relation Analysis for Compact SMT Encodings. In: Dillig, I., Tasiran, S. (eds.) CAV 2019. LNCS, vol. 11561, pp. 355–365. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-25540-4_19
21. Harmim, D., Marcin, V., Pavela, O.: Scalable static analysis using facebook Infer. I, VI-B (2019)
22. Havelund, K.: Using Runtime Analysis to Guide Model Checking of Java Programs. In: Havelund, K., Penix, J., Visser, W. (eds.) SPIN 2000. LNCS, vol. 1885, pp. 245–264. Springer, Heidelberg (2000). <https://doi.org/10.1007/10722468.15>
23. He, F., Sun, Z., Fan, H.: Satisfiability modulo ordering consistency theory for multi-threaded program verification. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, pp. 1264–1279 (2021). <https://doi.org/10.1145/3453483.3454108>
24. He, F., Sun, Z., Fan, H.: Deagle: An SMT-based verifier for multi-threaded programs (Competition contribution). In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems, pp. 424–428. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-99527-0_25
25. Herlihy, M., Shavit, N., Luchangco, V., Spear, M.: The art of multiprocessor programming, Newnes (2020)
26. Hoare, C.A.R.: Communicating sequential processes. *Commun. ACM* **21**(8), 666–677 (1978). <https://doi.org/10.1145/359576.359585>
27. Holt, R.C.: Some deadlock properties of computer systems. *ACM Comput. Surv. (CSUR)* **4**(3), 179–196 (1972). <https://doi.org/10.1145/356603.356607>
28. Huang, J.: UFO: predictive concurrency use-after-free detection. In: Proceedings of the 40th International Conference on Software Engineering, pp. 609–619 (2018). <https://doi.org/10.1145/3180155.3180225>

29. Huang, J., Meredith, P.O., Rosu, G.: Maximal sound predictive race detection with control flow abstraction. In: Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 337–348 (2014). <https://doi.org/10.1145/2594291.2594315>
30. Inverso, O., Nguyen, T.L., Fischer, B., La Torre, S., Parlato, G.: Lazy-CSeq: A context-bounded model checking tool for multi-threaded C-programs. In: 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 807–812. IEEE (2015). <https://doi.org/10.1109/ASE.2015.108>
31. Joshi, P., Park, C.S., Sen, K., Naik, M.: A randomized dynamic program analysis technique for detecting real deadlocks. ACM Sigplan Notices **44**(6), 110–120 (2009). <https://doi.org/10.1145/1542476.1542489>
32. Kalhauge, C.G., Palsberg, J.: Sound deadlock prediction. Proc. ACM Program. Lang. **2**(OOPSLA), 1–29 (2018). <https://doi.org/10.1145/3276516>
33. Kini, D., Mathur, U., Viswanathan, M.: Dynamic race prediction in linear time. ACM SIGPLAN Notices **52**(6), 157–170 (2017). <https://doi.org/10.1145/3062341.3062374>
34. Kroening, D., Poetzl, D., Schrammel, P., Wachter, B.: Sound static deadlock analysis for C/Pthreads. In: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, pp. 379–390 (2016). <https://doi.org/10.1145/2970276.2970309>
35. Kroening, D., Tautschnig, M.: CBMC – C Bounded Model Checker. In: Ábrahám, E., Havelund, K. (eds.) TACAS 2014. LNCS, vol. 8413, pp. 389–391. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-642-54862-8_26
36. Ponce-de León, H., Furbach, F., Heljanko, K., Meyer, R.: BMC with memory models as modules. In: 2018 Formal Methods in Computer Aided Design (FMCAD), pp. 1–9. IEEE (2018). <https://doi.org/10.23919/FMCAD.2018.8603021>
37. Love, R.: Linux kernel development, Pearson Education (2010)
38. Lu, L., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H., Lu, S.: A study of linux file system evolution. ACM Trans. Storage (TOS) **10**(1), 1–32 (2014). <https://doi.org/10.1145/2560012>
39. Lu, S., Park, S., Seo, E., Zhou, Y.: Learning from mistakes: a comprehensive study on real world concurrency bug characteristics. In: Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 329–339 (2008). <https://doi.org/10.1145/1346281.1346323>
40. Mathur, U., Pavlogiannis, A., Viswanathan, M.: Optimal prediction of synchronization-preserving races. Proc. ACM Program. Lang. **5**(POPL), 1–29 (2021). <https://doi.org/10.1145/3434317>
41. Pavlogiannis, A.: Fast, sound, and effectively complete dynamic race prediction. Proc. ACM Program. Lang. **4**(POPL), 1–29 (2019). <https://doi.org/10.1145/3371085>
42. Qadeer, S., Wu, D.: KISS: keep it simple and sequential. In: ACM-SIGPLAN Symposium on Programming Language Design and Implementation (2004). <https://doi.org/10.1145/996841.996845>
43. Rabinovitz, I., Grumberg, O.: Bounded Model Checking of Concurrent Programs. In: Etesami, K., Rajamani, S.K. (eds.) CAV 2005. LNCS, vol. 3576, pp. 82–97. Springer, Heidelberg (2005). https://doi.org/10.1007/11513988_9
44. Roemer, J., Genç, K., Bond, M.D.: SmartTrack: efficient predictive race detection. In: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 747–762 (2020). <https://doi.org/10.1145/3385412.3385993>

45. Sales, E., Inverso, O., Tuosto, E.: Accurate static data race detection for C. In: International Symposium on Formal Methods, pp. 443–462. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-71162-6_23
46. Samak, M., Ramanathan, M.K.: Multithreaded test synthesis for deadlock detection. In: Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications, pp. 473–489 (2014). <https://doi.org/10.1145/2660193.2660238>
47. Samak, M., Ramanathan, M.K.: Trace driven dynamic deadlock detection and reproduction. ACM SIGPLAN Notices **49**(8), 29–42 (2014). <https://doi.org/10.1145/2692916.2555262>
48. Șerbănuță, T.F., Chen, F., Roșu, G.: Maximal Causal Models for Sequentially Consistent Systems. In: Qadeer, S., Tasiran, S. (eds.) RV 2012. LNCS, vol. 7687, pp. 136–150. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-35632-2_16
49. Serebryany, K., Iskhodzhanov, T.: ThreadSanitizer: data race detection in practice. In: Proceedings of the workshop on binary instrumentation and applications, pp. 62–71 (2009). <https://doi.org/10.1145/1791194.1791203>
50. Silberschatz, A., Galvin, P.B., Gagne, G.: Operating system concepts essentials, Wiley Publishing (2013)
51. Smaragdakis, Y., Evans, J., Sadowski, C., Yi, J., Flanagan, C.: Sound predictive race detection in polynomial time. ACM Sigplan Notices **47**(1), 387–400 (2012). <https://doi.org/10.1145/2103621.2103702>
52. Sorrentino, F.: PICKLOCK: A Deadlock Prediction Approach under Nested Locking. In: Fischer, B., Geldenhuys, J. (eds.) SPIN 2015. LNCS, vol. 9232, pp. 179–199. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-23404-5_13
53. Stallings, W.: Operating Systems: Internals and Design Principles, 9/e. Pearson IT Certification (2018)
54. Sun, Z., Fan, H., He, F.: Consistency-preserving propagation for SMT solving of concurrent program verification. Proc. ACM Program. Lang. **6**(OOPSLA2), 929–956 (2022). <https://doi.org/10.1145/3563321>
55. Tanenbaum, A.S., Bos, H.: Modern operating systems. Pearson Education Inc., (2015)
56. Walia, E.: Operating system concepts, KHANNA PUBLISHING HOUSE (2002)
57. Wang, P., Han, Z., Sun, Z., He, F.: The artifacts of DEAGLE-DL (2026). <https://doi.org/10.5281/zenodo.19696864>
58. Yin, L., Dong, W., Liu, W., Li, Y., Wang, J.: YOGAR-CBMC: CBMC with Scheduling Constraint Based Abstraction Refinement. In: Beyer, D., Huisman, M. (eds.) TACAS 2018. LNCS, vol. 10806, pp. 422–426. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-89963-3_25
59. Yin, L., Dong, W., Liu, W., Wang, J.: Scheduling constraint based abstraction refinement for weak memory models. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering, pp. 645–655 (2018). <https://doi.org/10.1145/3238147.3238223>
60. Yin, L., Dong, W., Liu, W., Wang, J.: On scheduling constraint abstraction for multi-threaded program verification. IEEE Trans. Softw. Eng. **46**(05), 549–565 (2020). <https://doi.org/10.1109/TSE.2018.2864122>

61. Zhou, J., Silvestro, S., Liu, H., Cai, Y., Liu, T.: UNDEAD: Detecting and preventing deadlocks in production software. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 729–740. IEEE (2017). <https://doi.org/10.1109/ASE.2017.8115684>
62. Zou, M., Du, D., Dong, M., Chen, H.: Using dynamically layered definite releases for verifying the ReFFS file system. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), pp. 629–648 (2024). <https://www.usenix.org/conference/osdi24/presentation/zou>

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

